

Documentation

Smart Parking System

June 2026

1 Team members

1. Md Wasib Kamal Nirjon — ESP32 development, sensor setup, ultrasonic sensor testing, temperature and humidity sensor setup, flame sensor setup, and documentation.
2. Md Ratul Ahmed Alvi — Raspberry Pi development, backend server, data receiving logic, dashboard implementation, system integration, and Wi-Fi data transmission.

2 Introduction

The Internet of Things (IoT) refers to a network of physical devices that collect, exchange, and act on data without requiring direct human involvement. In our project, the IoT concept is applied to parking monitoring: embedded sensor nodes collect parking and environmental data, then send the information wirelessly to a central Raspberry Pi for processing and dashboard display.

A Wireless Sensor Network (WSN) in this context means multiple embedded processing units connected through Wi-Fi. The ESP32 development boards work as sensor nodes, reading ultrasonic sensors, temperature and humidity sensor, and a flame sensor. The Raspberry Pi works as the central hub that receives the sensor data, processes the current system status, and displays the results on a web-based dashboard.

The target application is a Smart Parking System. The goal is to detect whether parking slots are available or occupied and to monitor basic environmental and safety conditions around the parking area. The prototype focuses mainly on software development, real-time monitoring, wireless communication, and system integration.

3 Concept description

See the block diagram below (to be inserted as a figure). The system consists of two ESP32 sensor nodes connected wirelessly to one Raspberry Pi. One ESP32 node reads the ultrasonic sensors for parking slot detection. The second ESP32 node reads the temperature and humidity sensor and the flame sensor for environmental and safety monitoring. Both ESP32 nodes send structured sensor data to the Raspberry Pi over WiFi.

The main application of the prototype is real-time parking area monitoring. It operates continuously while the system is powered and updates the dashboard whenever new sensor data is received. The Raspberry Pi processes parking slot status, temperature, humidity, flame detection, and the overall system state.

Devices and components used: Raspberry Pi (main processing unit), two ESP32 development boards (sensor nodes), three ultrasonic sensors (parking slot detection), temperature and humidity sensor (environmental monitoring), flame sensor (safety monitoring), jumper wires, MicroSD card, and a Wi-Fi network or direct hotspot for inter-device communication.

4 Project/Team management

We follow an Agile-inspired approach with short iterative development cycles. Each iteration focuses on one part of the system: first testing the sensors individually, then connecting the

ESP32 nodes to Wi-Fi, then receiving data on the Raspberry Pi, and finally displaying the processed data on the dashboard.

Task breakdown: The project is split into two practical tracks. The ESP32 track focuses on sensor readings and wireless data transmission. The Raspberry Pi track focuses on backend processing, dashboard implementation, and system integration. Final integration and testing are handled jointly by both team members.

Roles: Md Ratul Ahmed Alvi — ESP32 subsystem lead (ultrasonic sensor setup, temperature and humidity sensor setup, flame sensor setup, sensor testing, Wi-Fi client, JSON packet transmission). Md Wasib Kamal Nirjon — Raspberry Pi subsystem lead (backend server, data receiver, dashboard, system status logic, documentation).

Alvi is responsible for connecting and testing the sensors with the ESP32 boards and preparing the data packets for wireless communication. Nirjon is responsible for setting up the Raspberry Pi software environment, receiving sensor data, processing the system state, and building the dashboard for real-time monitoring.

5 Technologies

The project is built on two complementary embedded platforms. On the ESP32 side, C/C++ with the Arduino framework is used to read sensors and send data over WiFi. On the Raspberry Pi side, Python is used to receive the data, process it, and run the backend for the dashboard.

- Sensors: three ultrasonic sensors for parking slot detection, one temperature and humidity sensor for environmental monitoring, and one flame sensor for safety monitoring.
- Communication protocols: WiFi for wireless data exchange between the ESP32 nodes and Raspberry Pi. Sensor values are transmitted using JSON packets through HTTP, MQTT, or a simple socket-based approach depending on the final implementation.
- Programming languages: C/C++ for the ESP32 nodes and Python 3 for the Raspberry Pi backend.
- Additional tools and libraries: Arduino IDE or PlatformIO for ESP32 programming, Flask or FastAPI for the Raspberry Pi server, and HTML/CSS/JavaScript for the dashboard interface.

6 Implementation

The system is composed of independent embedded sensor nodes connected to a central Raspberry Pi over a local Wi-Fi network. The ESP32 parking node reads the three ultrasonic sensors and determines whether each parking slot is available or occupied based on a configurable distance threshold. The ESP32 environment node reads temperature, humidity, and flame status. Each node sends its latest values to the Raspberry Pi as JSON data.

The Raspberry Pi runs a Python application with three main responsibilities: (1) receiving incoming data from the ESP32 nodes, (2) processing parking and safety status, and (3) updating the web dashboard. The dashboard displays slot availability, temperature, humidity, flame status, and overall system condition.

A state diagram (to be included as a figure) describes the system status logic on the Raspberry Pi. States: NORMAL (system operating without warning), SLOT_OCCUPIED (one or more parking slots are occupied), SLOT_AVAILABLE (one or more parking slots are free), ENVIRONMENT_ALERT (temperature or humidity value outside the selected normal range), and FIRE_ALERT (flame sensor detects possible flame or fire risk). Transitions are driven by sensor readings received from the ESP32 nodes.

Use case 1 — Parking slot monitoring: A vehicle enters or leaves a parking slot. The corresponding ultrasonic sensor detects the distance change. The ESP32 parking node sends the updated value to the Raspberry Pi, and the dashboard updates the slot status as available or occupied.

Use case 2 — Environmental monitoring: The temperature and humidity sensor measures the current environment. The ESP32 environment node sends these values to the Raspberry Pi, where they are displayed on the dashboard and compared with selected normal ranges.

Use case 3 — Flame alert: The flame sensor detects possible flame or fire risk. The ESP32 environment node sends the flame status to the Raspberry Pi. The Raspberry Pi changes the system state to FIRE_ALERT and the dashboard displays a warning status.

7 Results

Results will be documented here upon completion of implementation. Expected outcomes: successful detection of three parking slot statuses using ultrasonic sensors, successful temperature and humidity monitoring, successful flame detection, stable WiFi communication between ESP32 nodes and Raspberry Pi, and successful real-time display of all values on the dashboard.

Screenshots of the running dashboard, serial monitor outputs from the ESP32 nodes, Raspberry Pi console output, sensor test results, and final integration test results will be included in this section after implementation and testing.

8 Sources/References

ESP32 Arduino Core documentation — <https://github.com/espressif/arduino-esp32>

Raspberry Pi documentation — <https://www.raspberrypi.com/documentation/>

Flask documentation — <https://flask.palletsprojects.com/>

FastAPI documentation — <https://fastapi.tiangolo.com/>

Arduino documentation — <https://docs.arduino.cc/>